

ENSSAT

L A N N I O N

INFORMATIQUE
PHOTONIQUE
SYSTÈMES NUMÉRIQUES

Bertrand LE GAL

Laboratoire IRISA, UMR CNRS 5218

Département « D3 - Architecture »

Equipe projet INRIA « TARAN »

bertrand.le-gal@irisa.fr

<https://blegal.github.io>

Microelectronics
RISC-V core design



Université
de Rennes

École affiliée
IMT

Introduction

◎ What you should do

- ➔ Design a 32-bit processor core
- ➔ Validate the processor in simulation
- ➔ Prototyping the architecture on FPGA target
- ➔ Complexity study and optimization (Vivado & design compiler)

◎ Requirements

- ➔ Machine architecture (processor, pipeline, etc.)
- ➔ Digital architecture design
- ➔ VHDL description language
- ➔ C/ASM programming language

◎ Skills developed

- ➔ Digital architecture design
- ➔ VHDL modeling language
- ➔ Test and verification



◎ Introduction

- ➔ RISC-V presentation
- ➔ Processor architectures
- ➔ ASM language

◎ Preliminary work

- ➔ Analysis of the RISC-V instruction set
- ➔ Specification of the overall architecture
- ➔ Specification of the various entities

◎ Development and validation

- ➔ Description in VHDL language
- ➔ Validation of various modules

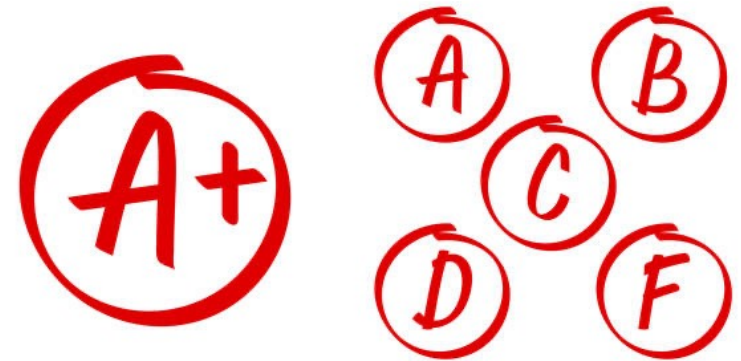
◎ ASIC core implementation ?

- ➔ Prototyping on FPGA board
- ➔ Implementation in ASIC technology



◎ Evaluation of the sequence

- ➔ Classroom-based
- ➔ VHDL source codes
- ➔ Implemented features
- ➔ FPGA-based core deployment
- ➔ Brief (pdf) report



◎ An ISA specification but...

- ➔ Different possible architectures
- ➔ Different ways of writing VHDL
- ➔ Different extensions
- ➔ Etc.

◎ Je vais vous challenger (markdown)

- ➔ Fonctionnalités décrites
- ➔ Fonctionnalités implantées
- ➔ Caractéristiques (complexité hardware, critical path)



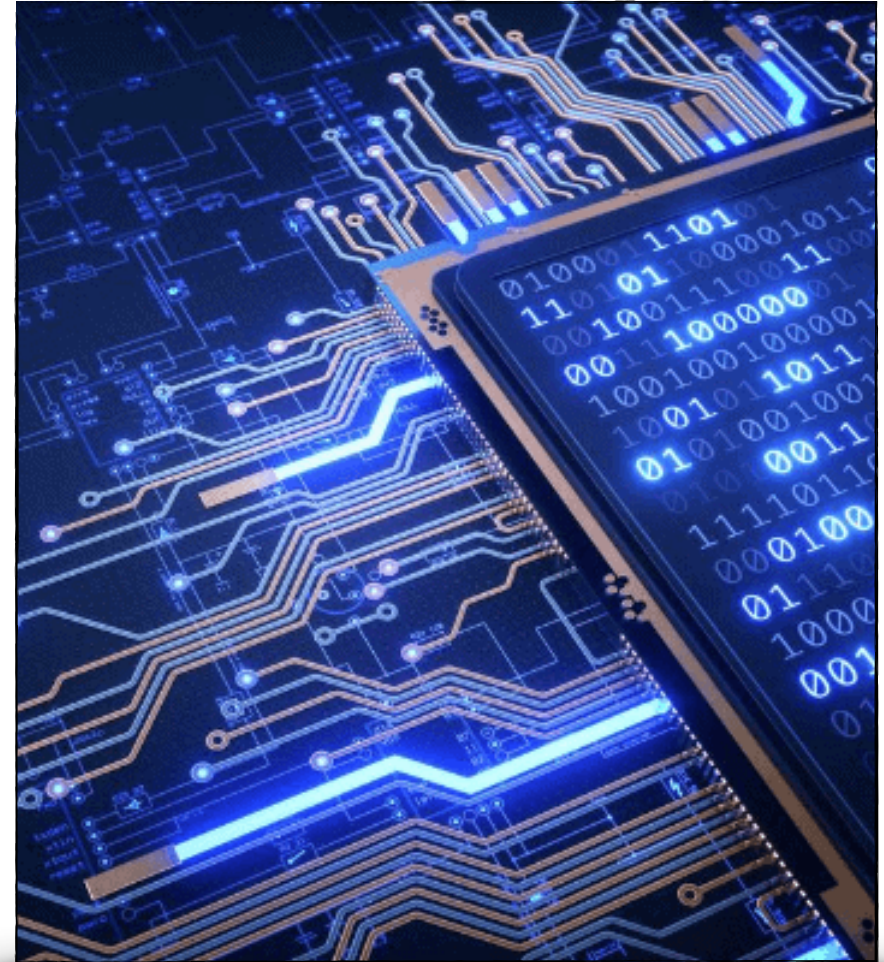
◎ Dedicated architectures

- ➔ Described at RTL level (mainly)
- ➔ ASIC and FPGA implemented
- ➔ Area efficient
- ➔ Energy efficient
- ➔ Throughput efficient

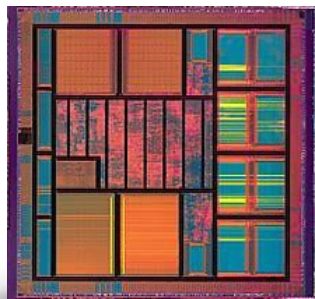
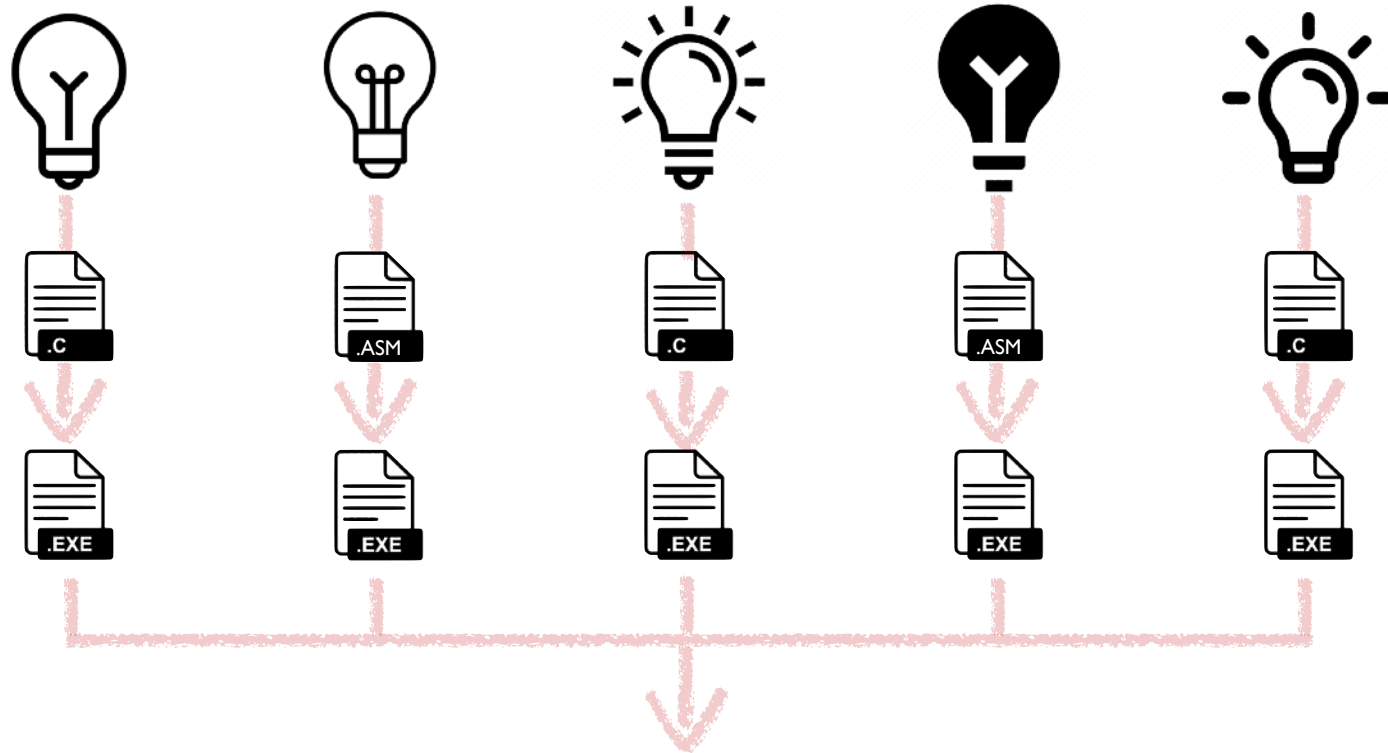
◎ However...

- ➔ Low flexibility level
- ➔ Complex to design
- ➔ Long development cycles
- ➔ Engineering skills

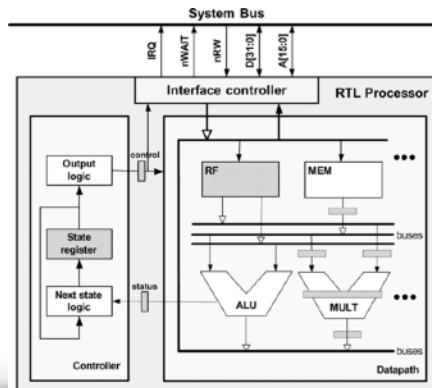
◎ Not always adapted to applications constraints



Les architectures programmables

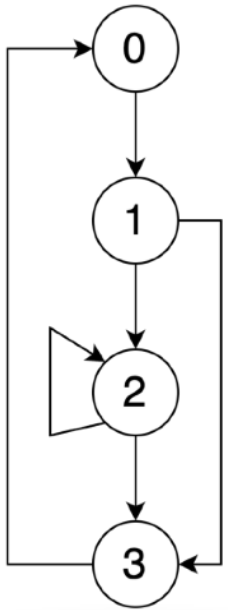


ASIC

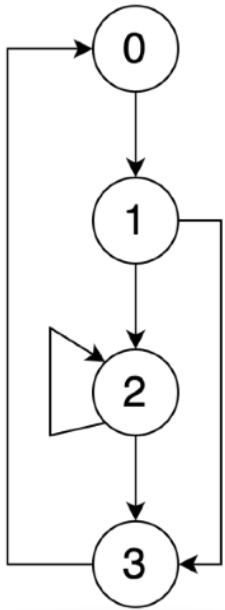


FPGA





Let's try with a great idea !
We want to execute a GCD computation...

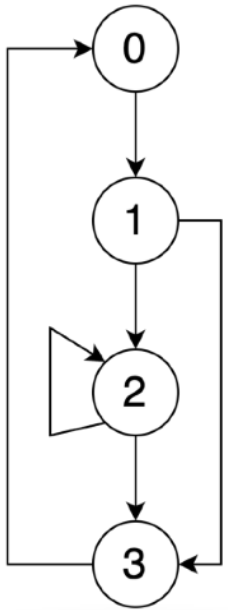


```

1  int main()
2  {
3      int a = 76152;
4      int b = 9865;
5
6      while( a != b )
7      {
8          if( a > b )
9              a -= b;
10         else
11             b -= a;
12     }
13
14     return a;
15 }
16

```

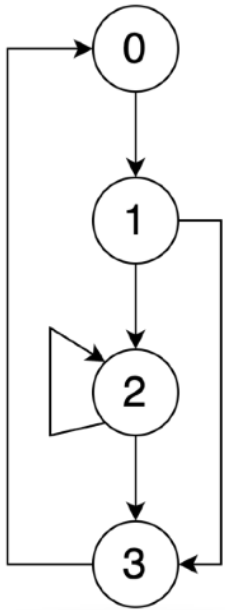
The « C »
description of the
GCD algorithm



The assembler program
executing the GCD algorithm.
ASM code automatically
generated by GCC

```

13      00000010 <main>:
14      10:      lui a5,0x2
15      14:      lui a0,0x13
16      18:      add a5,a5,1673
17      1c:      add a0,a0,-1672
18      20:      bge a5,a0,30 <main+0x20>
19      24:      sub a0,a0,a5
20      28:      bne a0,a5,20 <main+0x10>
21      2c:      ret
22      30:      sub a5,a5,a0
23      34:      bne a0,a5,20 <main+0x10>
24      38:      ret
  
```

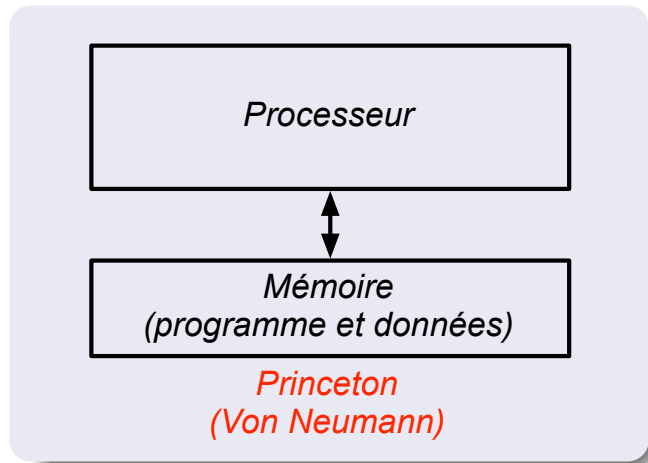


The binary program seen and
executed by the RISC-V
processor core
(produced by GCC AS)

```

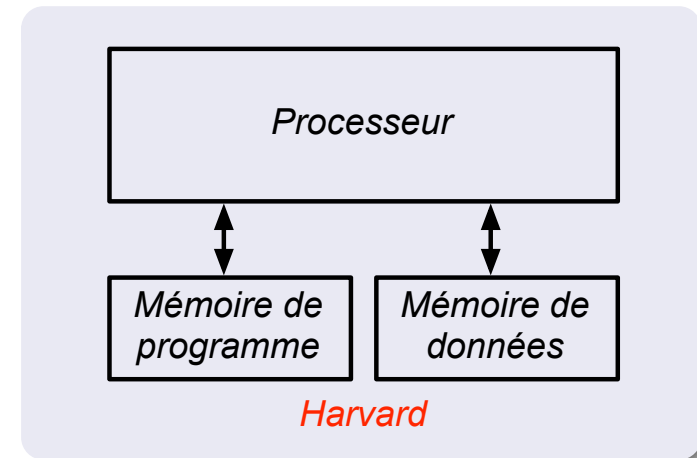
13      00000010 <main>:
14          10:  000027b7
15          14:  00013537
16          18:  68978793
17          1c:  97850513
18          20:  00a7d863
19          24:  40f50533
20          28:  fef51ce3
21          2c:  00008067
22          30:  40a787b3
23          34:  fef516e3
24          38:  00008067
  
```

Processor architectures

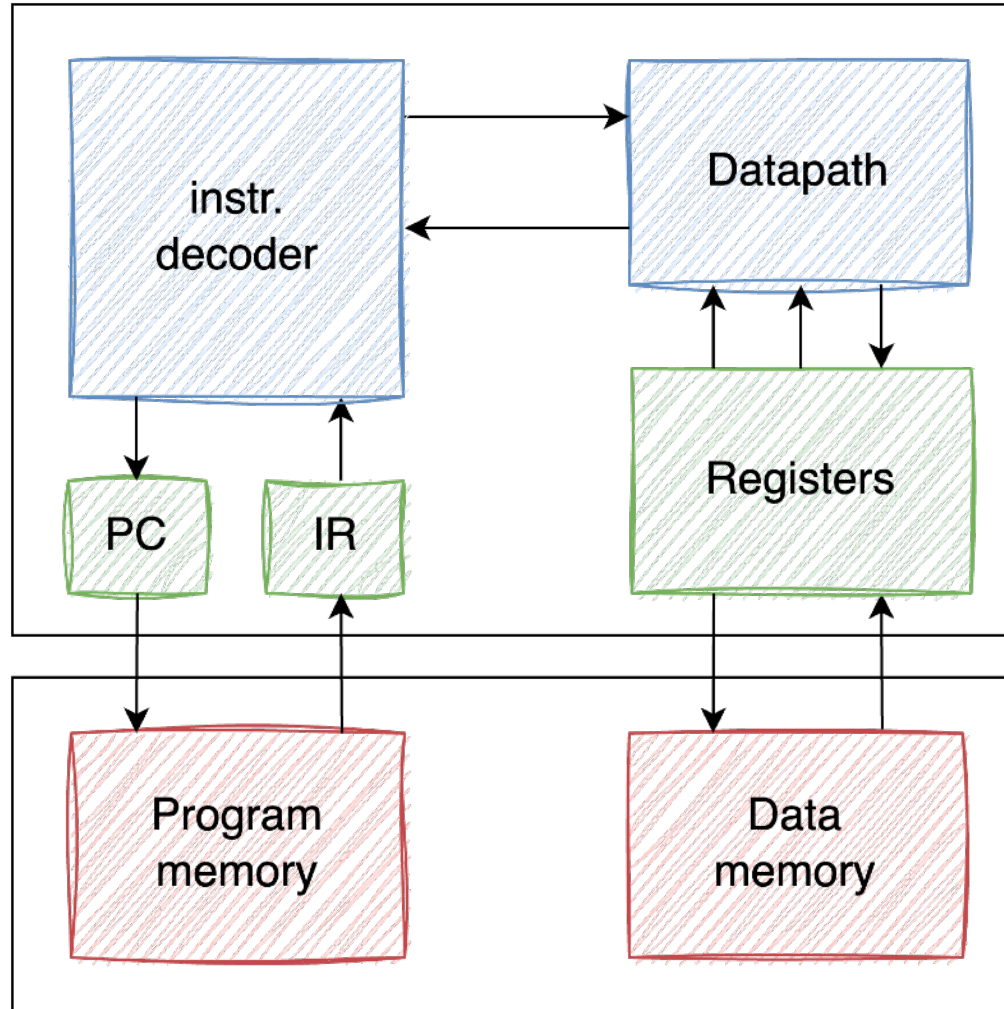


The processor has only one connection to memory. Instructions and data are mixed in memory. Reduces interconnections but limits performance (pipeline).

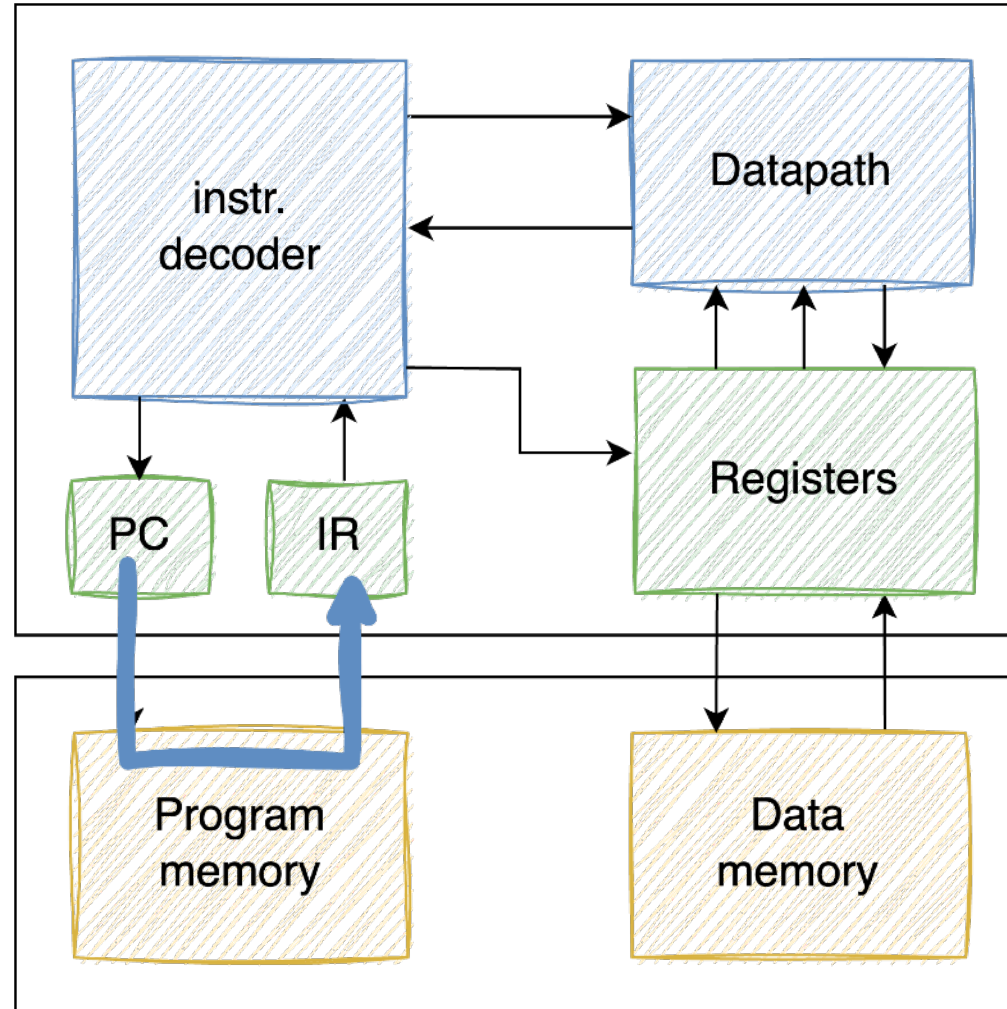
The processor has two memories. Instructions and data are separated. This allows for a pipeline (II=1), but has greater hardware complexity.



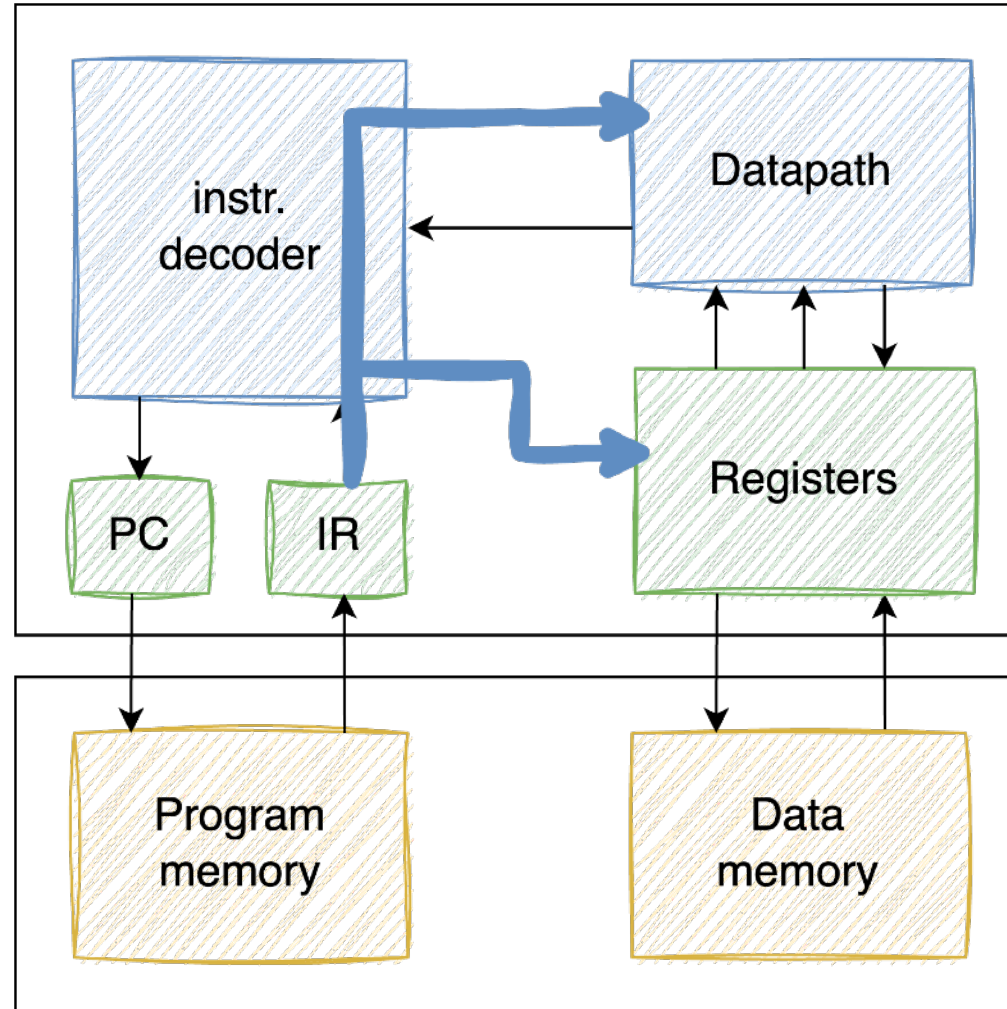
Simplified architecture behavior



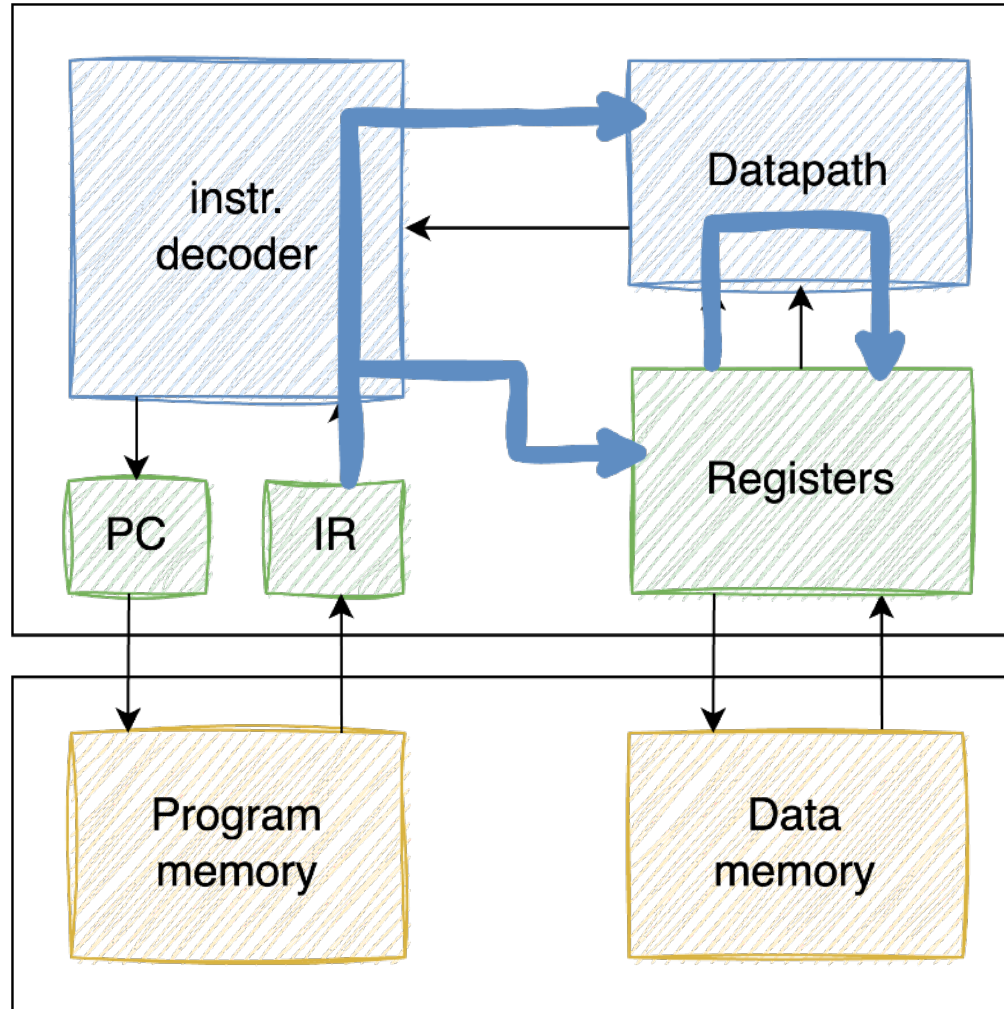
Simplified architecture behavior



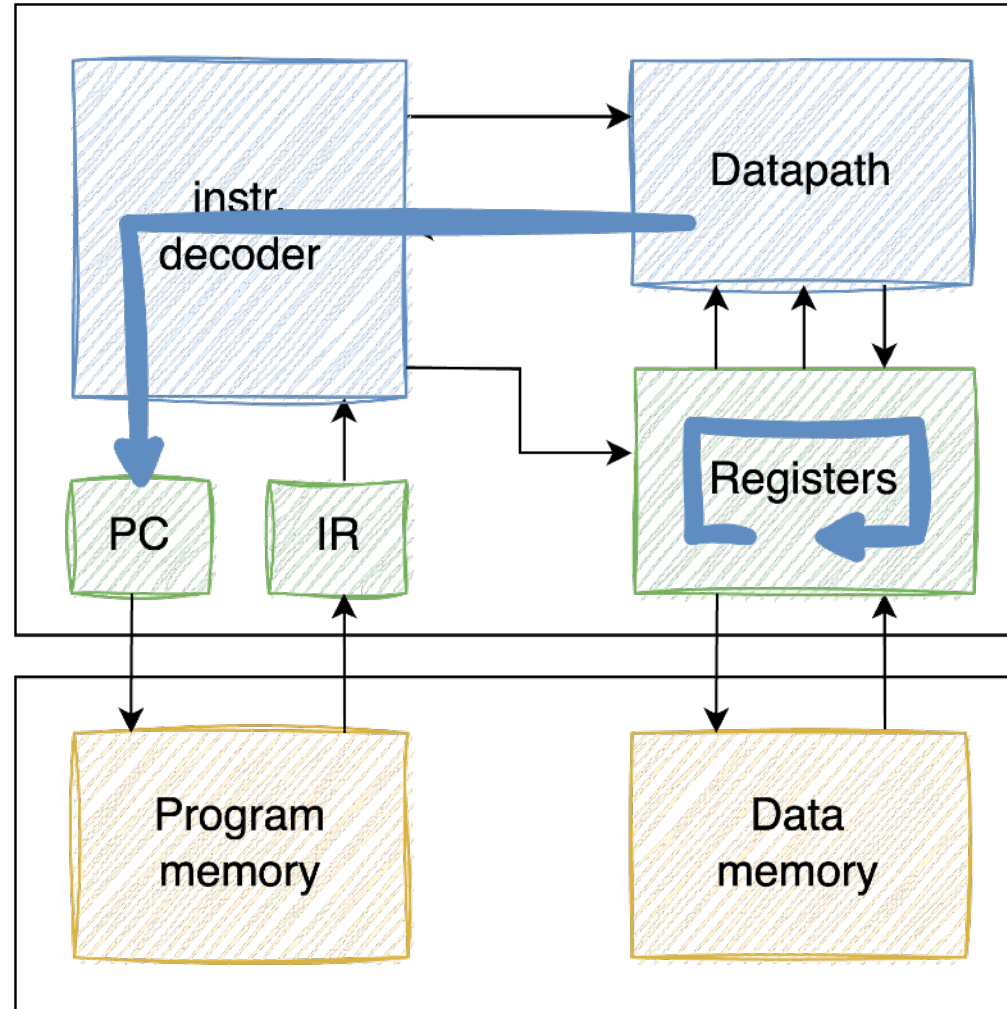
Simplified architecture behavior



Simplified architecture behavior



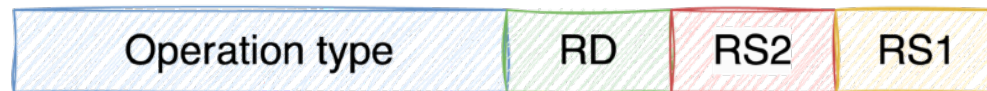
Simplified architecture behavior



Processor ISA



$[RD] \leq [RS1] \text{ OPERATION } [RS2]$



```
00000054 <print_string>:
54: add sp,sp,-16 # 1fff0 <val.0+0xffdc>
58: sw s0,8(sp)
5c: sw ra,12(sp)
60: mv s0,a0
64: lbu a0,0(a0)
68: beqz a0,7c <print_string+0x28>
6c: jal 3c <putchar>
70: lbu a0,1(s0)
74: add s0,s0,1
78: bnez a0,6c <print_string+0x18>
7c: lw ra,12(sp)
80: lw s0,8(sp)
84: add sp,sp,16
88: ret
```

```
00000054 <print_string>:
54: ff010113
58: 00812423
5c: 00112623
60: 00050413
64: 00054503
68: 00050a63
6c: fd1ff0ef
70: 00144503
74: 00140413
78: fe051ae3
7c: 00c12083
80: 00812403
84: 01010113
88: 00008067
```

◎ CPU ISAs ~= human languages

- ➔ Each processor has its own language
- ➔ Not so fun for software designers
- ➔ Not so fun for compiler designers

◎ ISA is composed of **P** instructions

- ➔ Instruction equals « basic » processor operation
- ➔ Used to generate control signals in the architecture
- ➔ Often coded on 32 bits

◎ ISA impacts on

- ➔ Performances of program execution
- ➔ Program memory footprint
- ➔ Hardware design



◎ RISC

- ➔ Reduced Instruction Set Computer

◎ Advantages

- ➔ Low hardware complexity
- ➔ Low power consumption

◎ Drawbacks

- ➔ Low instruction densities
- ➔ High compiler complexity

◎ Some well-know architectures

- ➔ DEC Alpha (64 bits) - 1992
- ➔ MIPS architecture (32/64 bits) - 1984
- ➔ SPARC (32/64 bits) - 1987
- ➔ ARM (32/64 bits) - 1985

Aspect	RISC
Instructions Per Cycle	Small and fixed length
Instruction Complexity	Simple and standardised
Instruction Execution	Single clock cycle
RAM Usage	Heavy use of RAM
Memory	Increased memory usage to store instructions
Cost	Higher Cost

◎ CISC

- ➔ Complex Instruction Set Computer

◎ Advantages

- ➔ High instruction densities
- ➔ High hardware efficiency
- ➔ Low compiler complexity
- ➔ Instructions with memory operands

◎ Drawbacks

- ➔ High hardware complexity

◎ Some well-know architectures

- ➔ INTEL x86 family (more than 800 instr.)
- ➔ Motorola 68000
- ➔ PowerPC (more than 230 instr.)

Aspect	CISC
Instructions Per Cycle	Large and variable length
Instruction Complexity	Complex and versatile
Instruction Execution	Several clock cycles
RAM Usage	More efficient use of RAM
Memory	Memory efficient coding
Cost	Cheaper than RISC

© Most ISA are patented

- ➔ For instance, nobody can design an ARM compliant core !
- ➔ The binary programs are limited to an silicon provider
- ➔ Important porting cost between CPU solutions (software, libraries, compilers)



© Open-source hardware

- ➔ Like Linux 20 years ago...
- ➔ Portable and open source solutions
- ➔ Not so easy due to \$\$\$ (hardware, software, tools)

© Main motivations

- ➔ Sovereignty
- ➔ Flexibility & compatibility



RISC-V processors

© Motivations

- ➔ Compatible processor cores from IoT to HPC
- ➔ ISA only is standardized (not arch.)
- ➔ Open-source ISA for academics
- ➔ Fluent compiler and library development

RISC-V®

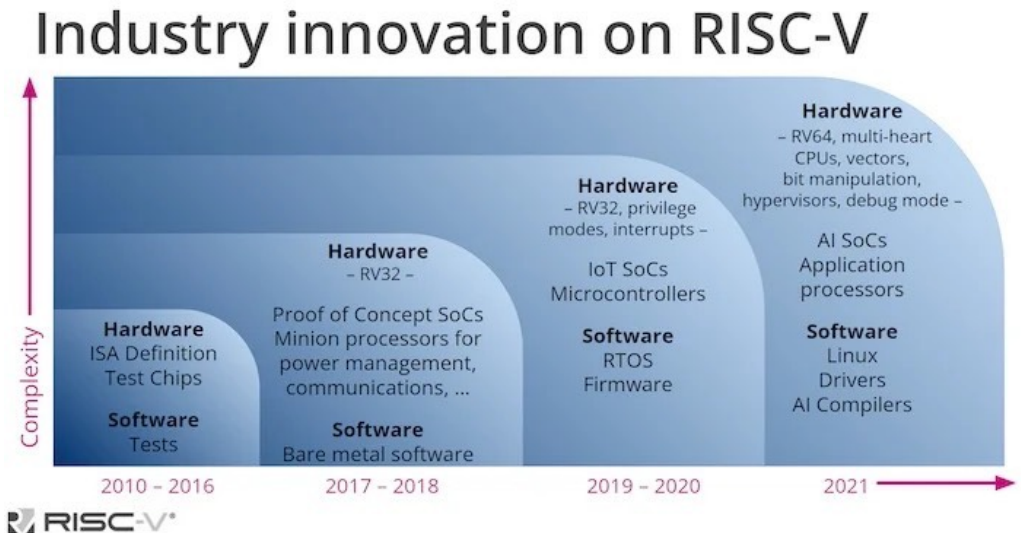


© Project background

- ➔ University of California, Berkeley
- ➔ 2014 (open hardware)
- ➔ Generic ISA specification
- ➔ Industrials & academics

© Extension based ISA

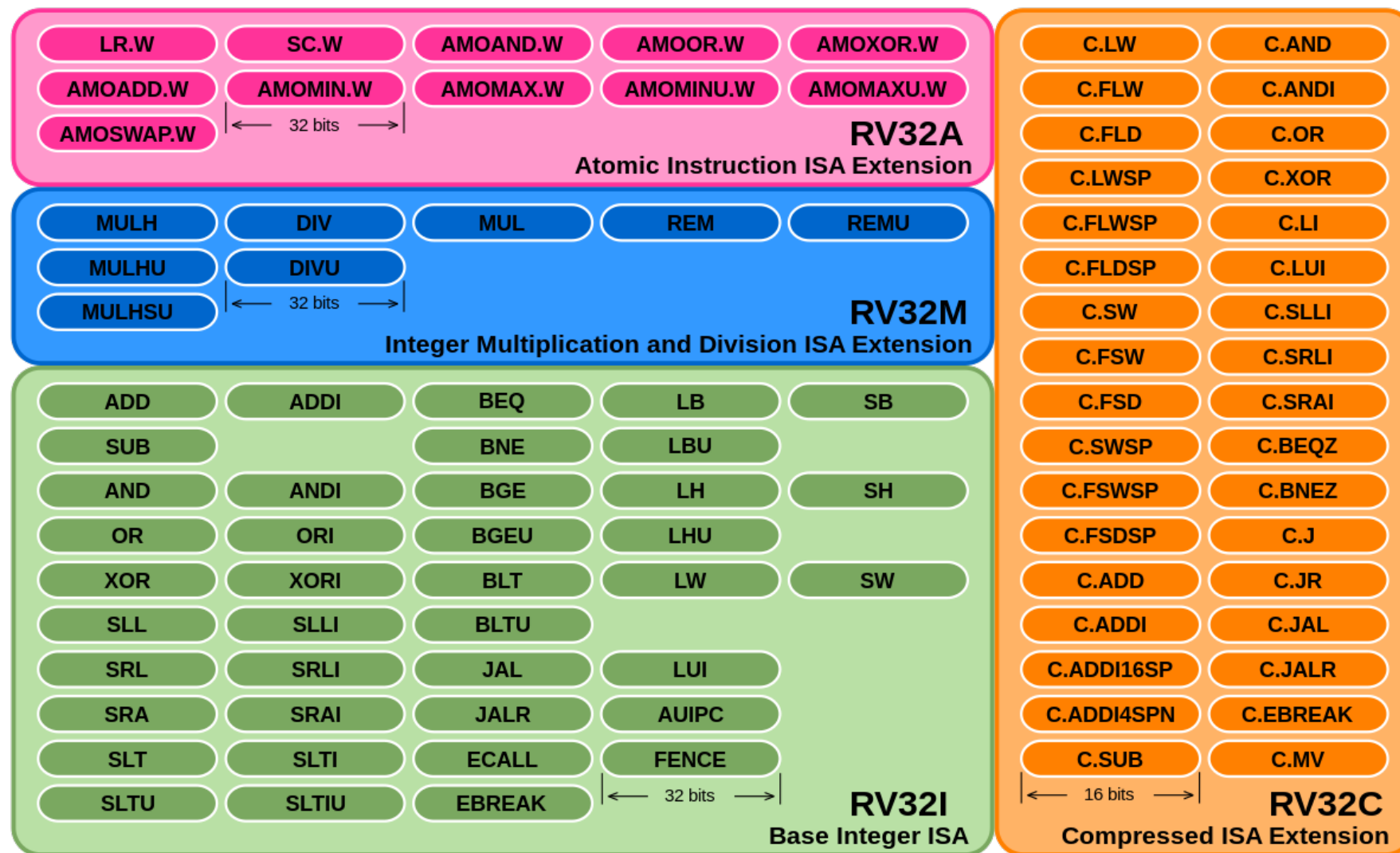
- ➔ RISC-V base (RV32i - 32b)
- ➔ Instruction extensions (E, M...)
- ➔ Width extensions (32b to 128b)



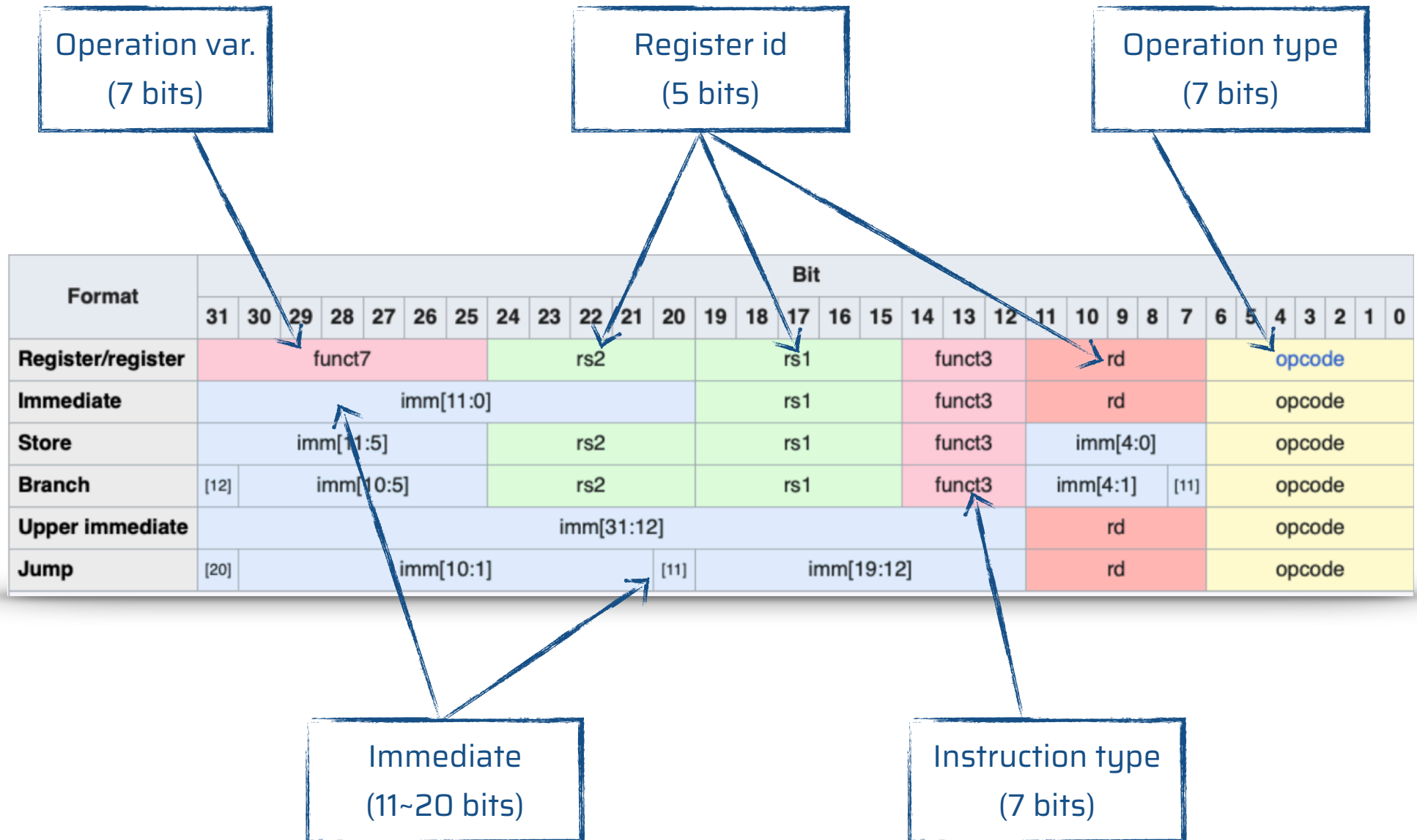


<https://riscv.org/members/>

RV32IMAC



<https://en.wikipedia.org/wiki/RISC-V>



© RV32I ISA

- ➔ 32-bit instruction
- ➔ 32 general purpose registers
- ➔ ~51 possible instructions

© 6 instruction formats

- ➔ {R, I, S, B, U, J}-type

32-bit RISC-V instruction formats

Format	Bit																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Register/register	funct7							rs2				rs1				funct3				rd				opcode								
Immediate	imm[11:0]												rs1				funct3				rd				opcode							
Store	imm[11:5]							rs2				rs1				funct3				imm[4:0]				opcode								
Branch	[12]	imm[10:5]						rs2				rs1				funct3				imm[4:1]		[11]	opcode									
Upper immediate	imm[31:12]																				rd				opcode							
Jump	[20]	imm[10:1]										[11]	imm[19:12]								rd				opcode							

R-type instructions (register)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Arithmetic and logic instructions between two registers

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0

I-type instructions (immediate)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Arithmetic and logic instructions between a register and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0

I-type instructions (immediate)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Load instructions involving two registers and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]

S-type instructions (store)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Store instructions involving two registers and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
sb	Store Byte	S	0100011	0x0		$M[rs1+imm][0:7] = rs2[0:7]$
sh	Store Half	S	0100011	0x1		$M[rs1+imm][0:15] = rs2[0:15]$
sw	Store Word	S	0100011	0x2		$M[rs1+imm][0:31] = rs2[0:31]$

I-type instructions (branch)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Conditional branch instructions involving two registers and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm

J-type instructions (long jump)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Jump instructions involving one or two register(s) and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm

I-type instructions (immediate)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Immediate load instruction involving a register and an immediate

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
lui	Load Upper Imm	U	0110111			rd = imm << 12
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)

I-type instructions (immediate)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Instructions used to interact with the system

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger

RV32i - Instruction groups vs opcode

Inst	Name	FMT	Opcode	funct3
add	ADD	R	0110011	0x0
sub	SUB	R	0110011	0x0
xor	XOR	R	0110011	0x4
or	OR	R	0110011	0x6
and	AND	R	0110011	0x7
sll	Shift Left Logical	R	0110011	0x1
srl	Shift Right Logical	R	0110011	0x5
sra	Shift Right Arith*	R	0110011	0x5
slt	Set Less Than	R	0110011	0x2
sltu	Set Less Than (U)	R	0110011	0x3
addi	ADD Immediate	I	0010011	0x0
xori	XOR Immediate	I	0010011	0x4
ori	OR Immediate	I	0010011	0x6
andi	AND Immediate	I	0010011	0x7
slli	Shift Left Logical Imm	I	0010011	0x1
srli	Shift Right Logical Imm	I	0010011	0x5
srai	Shift Right Arith Imm	I	0010011	0x5
slti	Set Less Than Imm	I	0010011	0x2
sltiu	Set Less Than Imm (U)	I	0010011	0x3

Inst	Name	FMT	Opcode	funct3
lb	Load Byte	I	0000011	0x0
lh	Load Half	I	0000011	0x1
lw	Load Word	I	0000011	0x2
lbu	Load Byte (U)	I	0000011	0x4
lhu	Load Half (U)	I	0000011	0x5
sb	Store Byte	S	0100011	0x0
sh	Store Half	S	0100011	0x1
sw	Store Word	S	0100011	0x2
beq	Branch ==	B	1100011	0x0
bne	Branch !=	B	1100011	0x1
blt	Branch <	B	1100011	0x4
bge	Branch ≥	B	1100011	0x5
bltu	Branch < (U)	B	1100011	0x6
bgeu	Branch ≥ (U)	B	1100011	0x7
jal	Jump And Link	J	1101111	
jalr	Jump And Link Reg	I	1100111	0x0
lui	Load Upper Imm	U	0110111	
auipc	Add Upper Imm to PC	U	0010111	
ecall	Environment Call	I	1110011	0x0
ebreak	Environment Break	I	1110011	0x0

◎ M-extension (multiply & divide)

- support for integer multiplication and division instructions

◎ A-extension (atomic)

- Atomic extension ease concurrent access to shared memory resources (multi-core)

◎ F-extension (float)

- support for single-precision floating-point arithmetic operations

◎ C-extension (compressed)

- compressed instruction format to reduce program size (like microcontrollers)

◎ B-extension (bitmanip)

- ease the execution of bit level computations (size, energy)

◎ Zbkb-extension (crypto)

- improve cryptographic application efficiency

https://five-embeddev.com/quickref/isa_ext.html



RISC-V processors have 32 general purpose registers (GPR)

<i>RISC-V Calling Convention</i>			
Register	ABI Name	Saver	Description
x0	zero	---	Hard-wired zero
x1	ra	Caller	Return address
x2	sp	Callee	Stack pointer
x3	gp	---	Global pointer
x4	tp	---	Thread pointer
x5-7	t0-2	Caller	Temporaries
x8	s0/fp	Callee	Saved register/frame pointer
x9	s1	Callee	Saved register
x10-11	a0-1	Caller	Function arguments/return values
x12-17	a2-7	Caller	Function arguments
x18-27	s2-11	Callee	Saved registers
x28-31	t3-t6	Caller	Temporaries
f0-7	ft0-7	Caller	FP temporaries
f8-9	fs0-1	Callee	FP saved registers
f10-11	fa0-1	Caller	FP arguments/return values
f12-17	fa2-7	Caller	FP arguments
f18-27	fs2-11	Callee	FP saved registers
f28-31	ft8-11	Caller	FP temporaries

Registers have precise usages specified for compatibility purpose between programs, libraries and compilers.

Playing with assembler

◎ Find the instructions required to implement the following « so simple » C codes

- ➡ NOP instruction ?
- ➡ `int R1 = 13;`
- ➡ `int R2 = 13; (another way)`
- ➡ `int* R3 = 1024;`
- ➡ `int R4 = (*R3);`
- ➡ `int R5 = R1 * R2 * R2 + R4 * R2 + R4;`
- ➡ `int R6 = (R5 > R4) ? -R5 : R4`

© Find out what these hexadecimal codes correspond to?

➔ 000027b7

➔ 68978793

➔ 00a7d863

➔ fef516e3

➔ 00008067

➔ fd1ff0ef

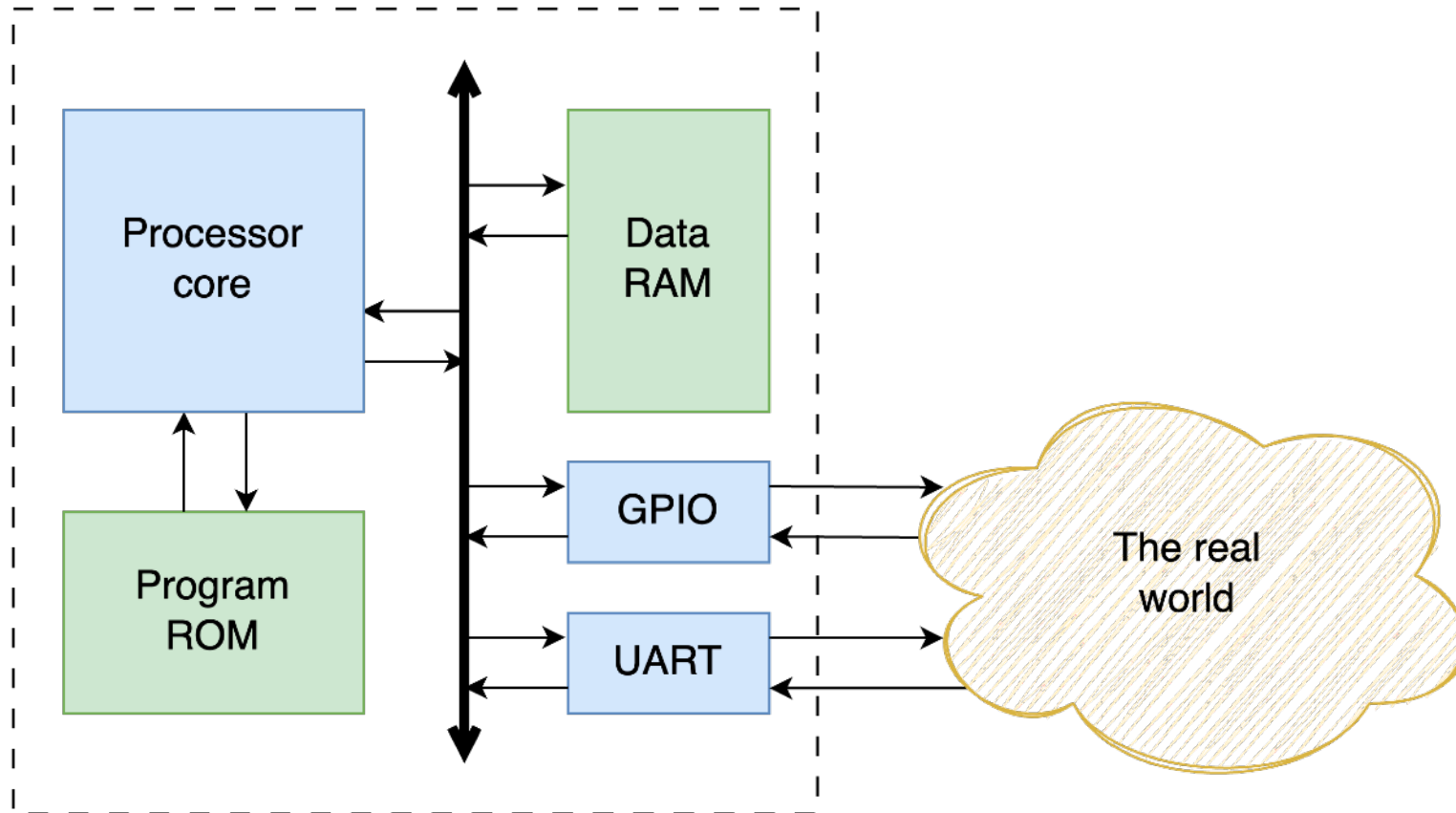
© Find out what these hexadecimal codes correspond to?

- ➔ 000027b7 lui a5,0x2
- ➔ 68978793 add a5,a5,1673
- ➔ 00a7d863 bge a5,a0,34 <main+0x20>
- ➔ fef516e3 bne a0,a5,24 <main+0x10>
- ➔ 00008067 ret
- ➔ fd1ff0ef jal 40 <putchar>

- © Write the binary representation of the instructions used in the previous slide (slide - 2)

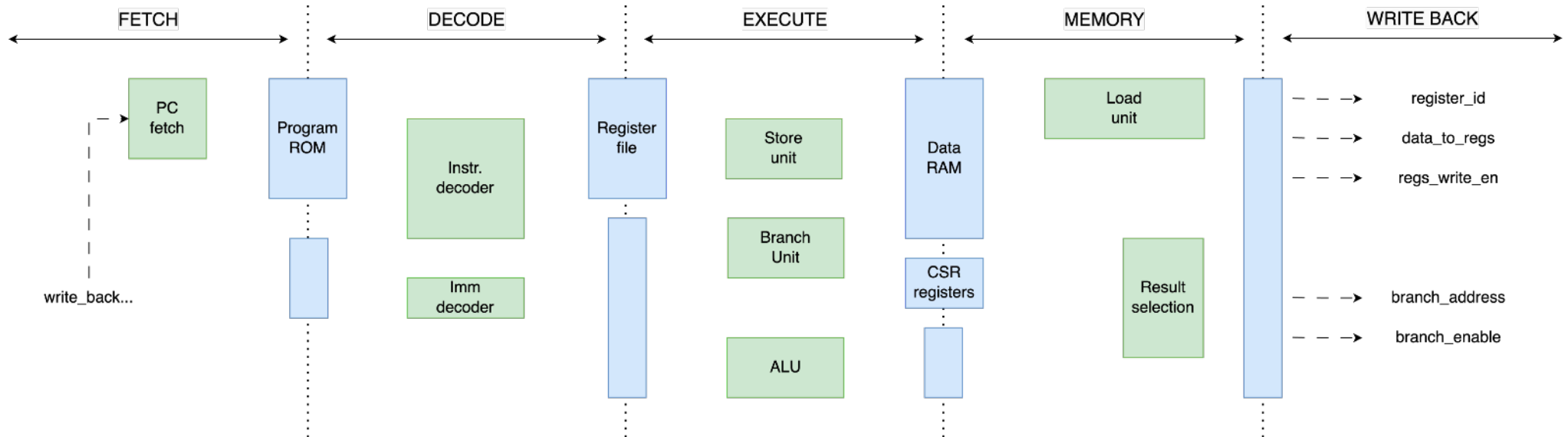
From ISA to RTL architecture

FPGA



The CPU core will be included in a system containing several basic peripherals. This system will interconnect the CPU with the rest of the system.

Coarse grain architecture



The different architecture slices are executed sequentially. At a given clock cycle, only one slice is active.

The architecture's "general" controller is a simple shift register that distributes an enable signal to the registers & memory blocks.

◎ You won't have to do everything !

- ➔ Sequential parts (except the memories) are provided by your teacher
- ➔ SoC design and its test bench are also delivered
- ➔ C/C++ examples are also available to ease your design and verification tasks.

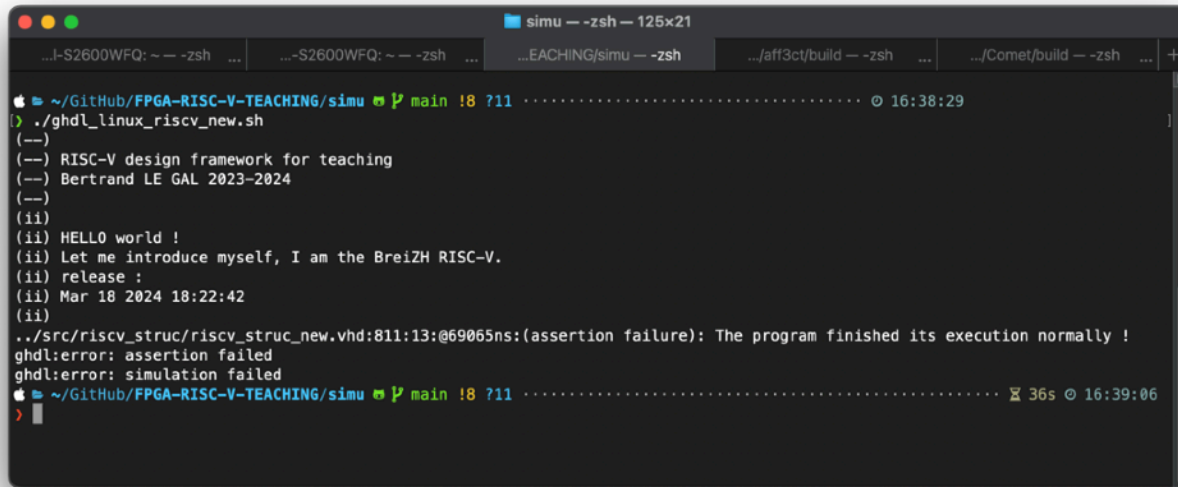
◎ It is important to

- ➔ Understand the behavior of the modules
- ➔ Write clear VHDL codes
- ➔ Test them correctly

◎ You ...

- ➔ should not share your VHDL codes !
- ➔ can share your VHDL testbenchs





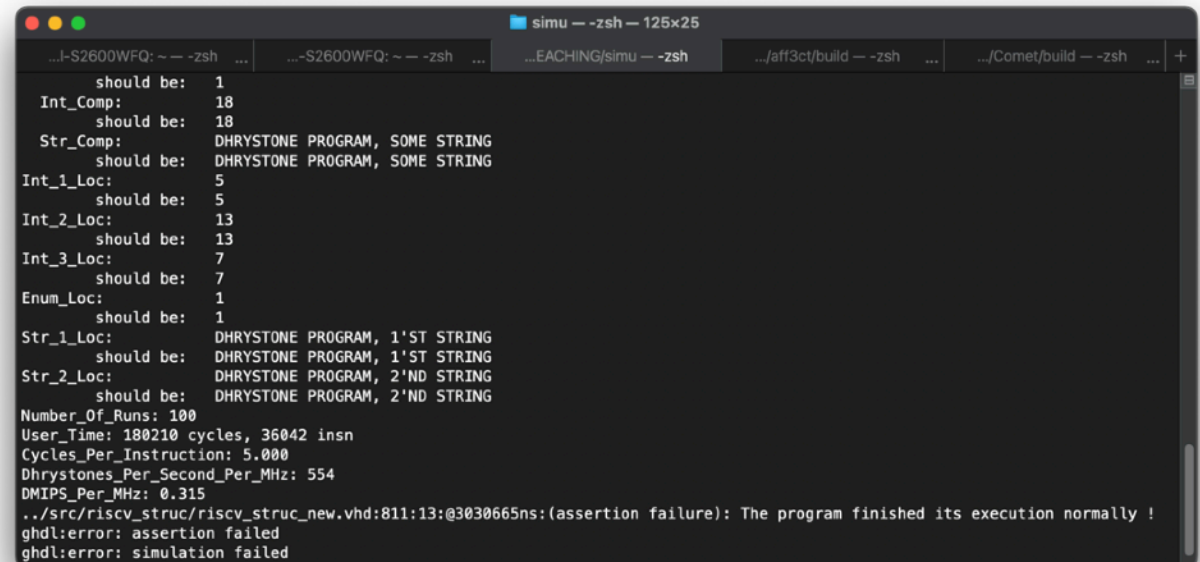
```

simu --zsh -- 125x21
...-S2600WFO: ~ --zsh ...
...EACHING/simu --zsh
.../aff3ct/build --zsh ...
.../Comet/build --zsh ...
~/GitHub/FPGA-RISC-V-TEACHING/simu main !8 ?11 16:38:29
./ghdl_linux_riscv_new.sh
(--)
(--) RISC-V design framework for teaching
(--) Bertrand LE GAL 2023-2024
(--)
(ii) HELLO world !
(ii) Let me introduce myself, I am the BreiZH RISC-V.
(ii) release :
(ii) Mar 18 2024 18:22:42
(ii)
../src/riscv_struc/riscv_struc_new.vhd:811:13:@69065ns:(assertion failure): The program finished its execution normally !
ghdl:error: assertion failed
ghdl:error: simulation failed
~/GitHub/FPGA-RISC-V-TEACHING/simu main !8 ?11 36s 16:39:06

```

The simulation scripts provided allow to simulate the SoC. It is then possible to get the UART output a the terminal.

This simulation approach also enable to run much more complex software codes such as Dhrystones one.



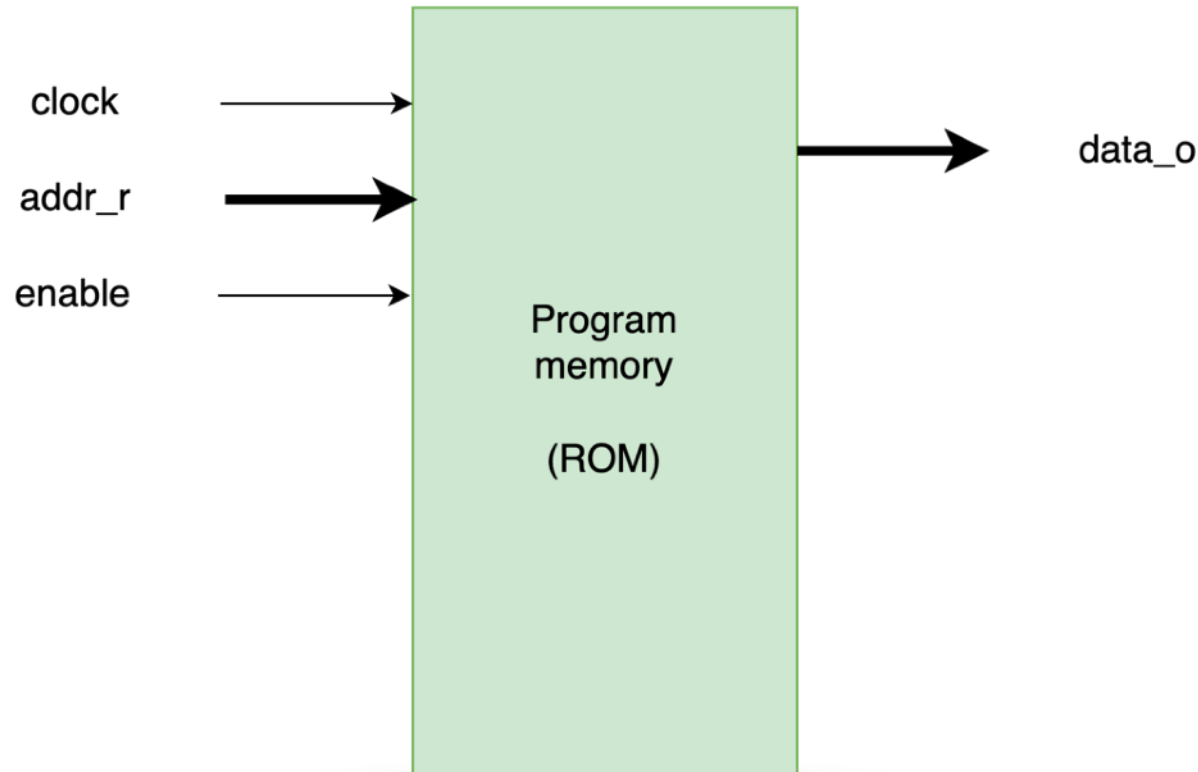
```

simu --zsh -- 125x25
...-S2600WFO: ~ --zsh ...
...EACHING/simu --zsh
.../aff3ct/build --zsh ...
.../Comet/build --zsh ...
should be: 1
Int_Comp: 18
should be: 18
Str_Comp: DHRYSTONE PROGRAM, SOME STRING
should be: DHRYSTONE PROGRAM, SOME STRING
Int_1_Loc: 5
should be: 5
Int_2_Loc: 13
should be: 13
Int_3_Loc: 7
should be: 7
Enum_Loc: 1
should be: 1
Str_1_Loc: DHRYSTONE PROGRAM, 1'ST STRING
should be: DHRYSTONE PROGRAM, 1'ST STRING
Str_2_Loc: DHRYSTONE PROGRAM, 2'ND STRING
should be: DHRYSTONE PROGRAM, 2'ND STRING
Number_Of_Runs: 100
User_Time: 180210 cycles, 36042 insn
Cycles_Per_Instruction: 5.000
Dhrystones_Per_Second_Per_MHz: 554
DMIPS_Per_MHz: 0.315
../src/riscv_struc/riscv_struc_new.vhd:811:13:@3030665ns:(assertion failure): The program finished its execution normally !
ghdl:error: assertion failed
ghdl:error: simulation failed

```

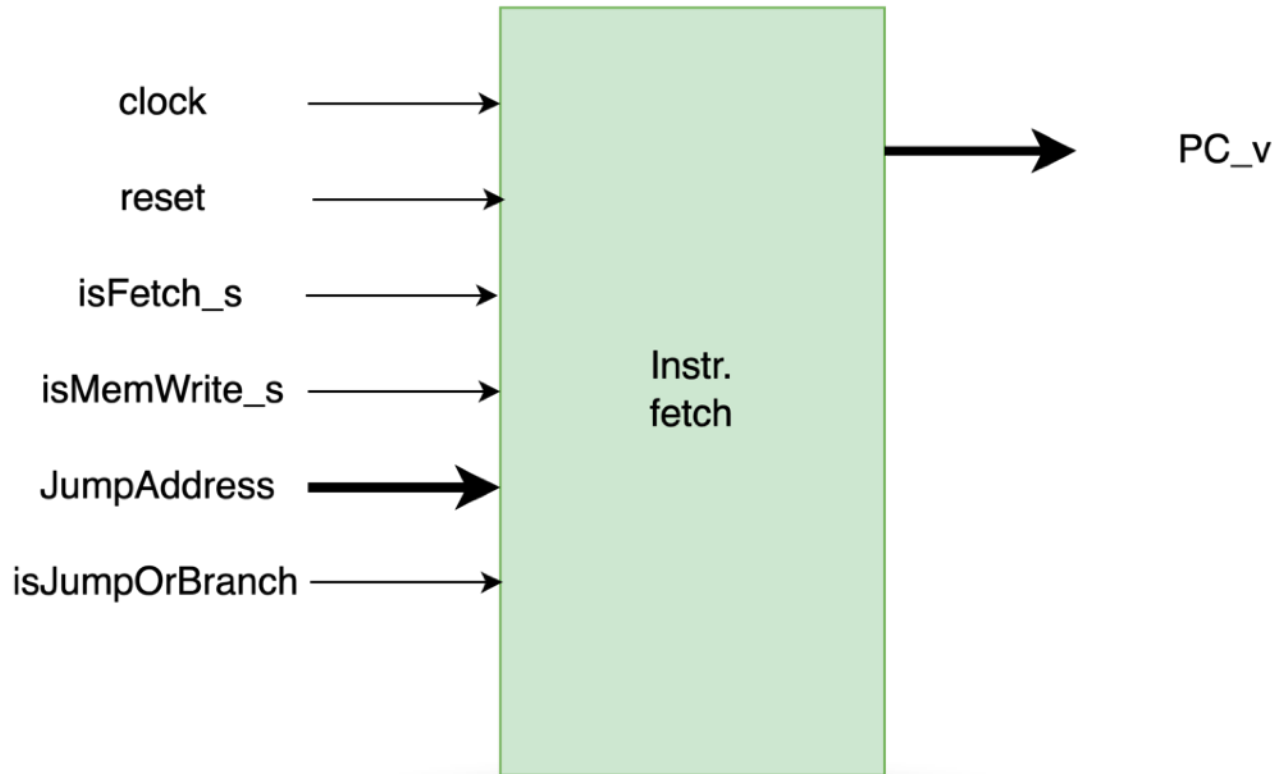
Specification of the RTL modules

The ROM module



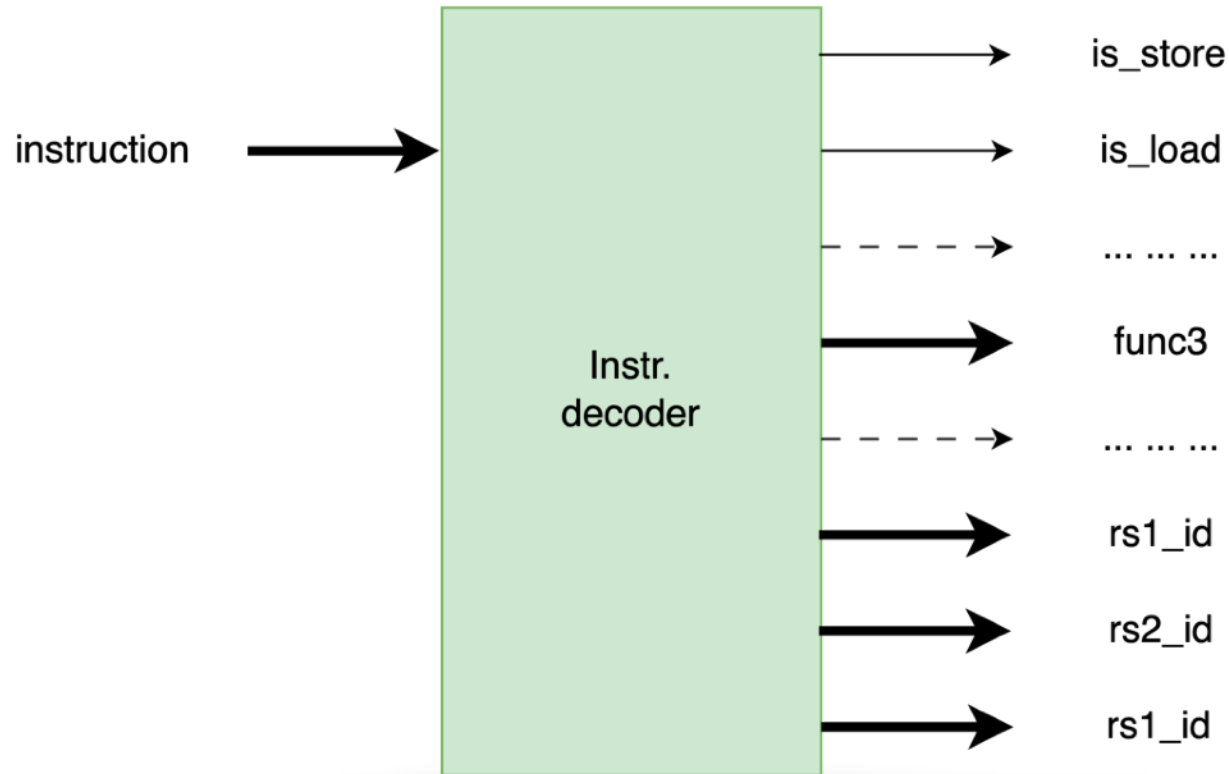
The instruction memory is read-only. The executed program is hard-loaded during synthesis. The executed instruction must be updated in the FETCH stage.

The fetch module



The FECTCH module is in charge of updating the PC value. In normal operation (isFetch), it will update by +4, otherwise, in the event of a branch or a jump instruction, it loads the value pre-calculated during the write-back stage.

The instruction decoder unit



Its function is to generate all the control signals that will pilot the architecture. Its definition must be consistent with the ISA specification.

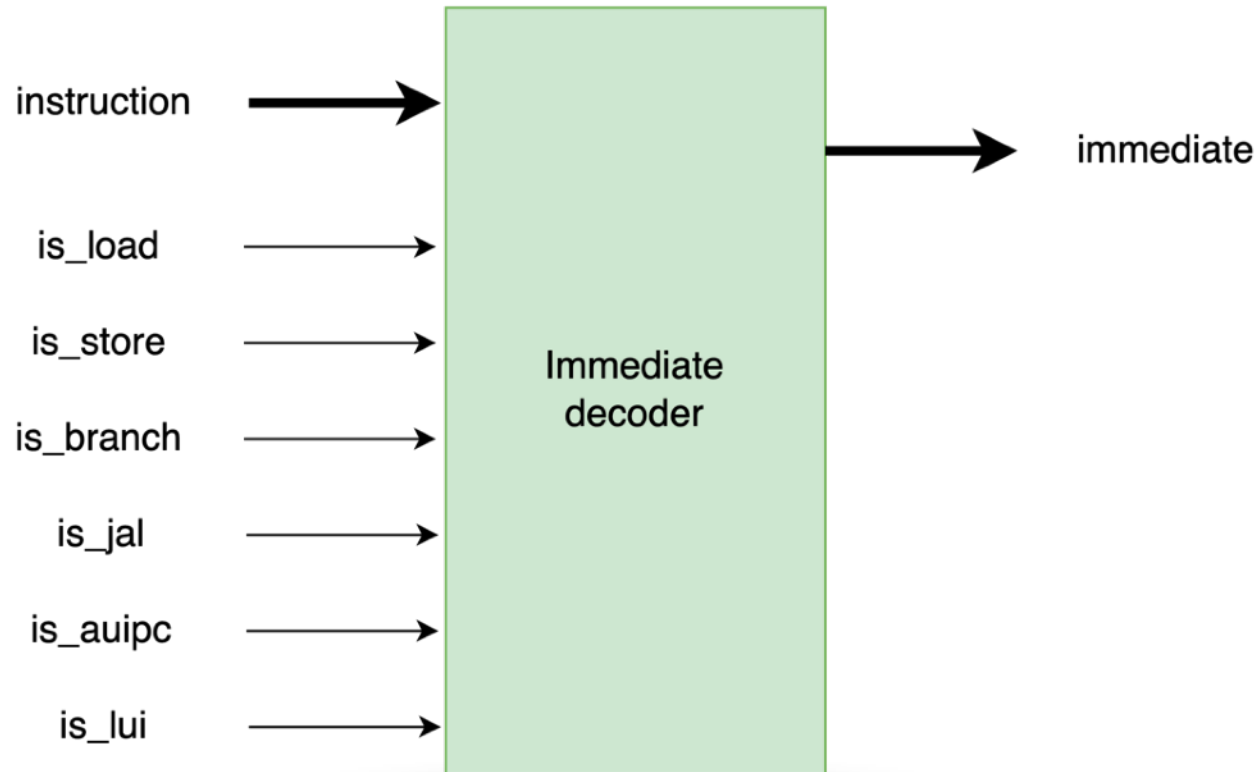
◎ Draw the internal architecture of the decoder

- ➔ Module input(s):
 - instr_i (32 bits);
- ➔ Module output(s)
 - isLoad_o, isStore_o, isALUreg_o, isBranch_o, isSYSTEM_o, isJAL_o, isJALR_o, isJALorJALR_o, isAuipc_o, isLui_o, isCSRrs_o, isEBreak_o (1 b)
 - isByte_o, isHalf_o (1 bit)
 - funct3_o (3 bits)
 - funct7_o (7 bits)
 - csrId_o (2 bits)
 - rs1_o, rs2_o, rdId_o (5 bits)

◎ Write a part of the VHDL description using a pen !

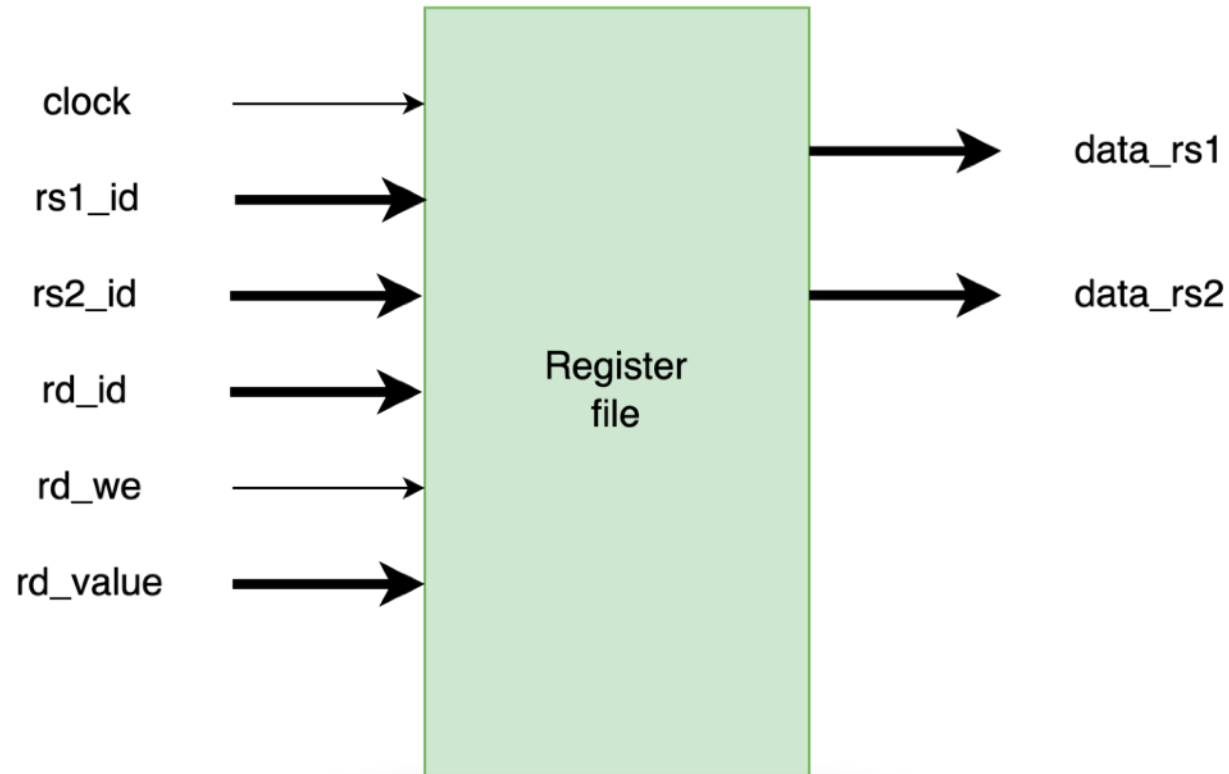


The immediate value decoder unit



Depending on the type of instruction, the decoding of the immediate data differs. Here, the choice is made to extract only one from the decoding stage. Immediate data is represented on 32 signed bits.

The register file module

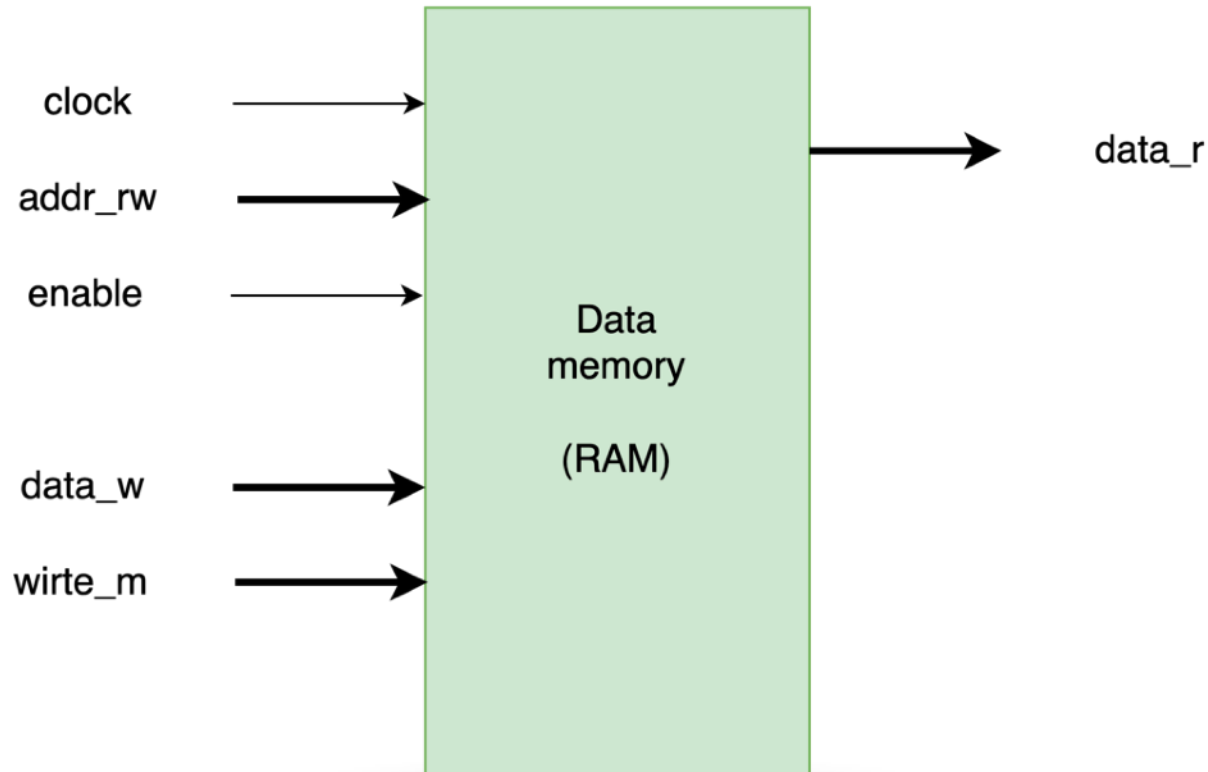


The register file is responsible for storing data in the 32 general registers. Writing to RD is conditional, as not all ISA instructions update RD register.



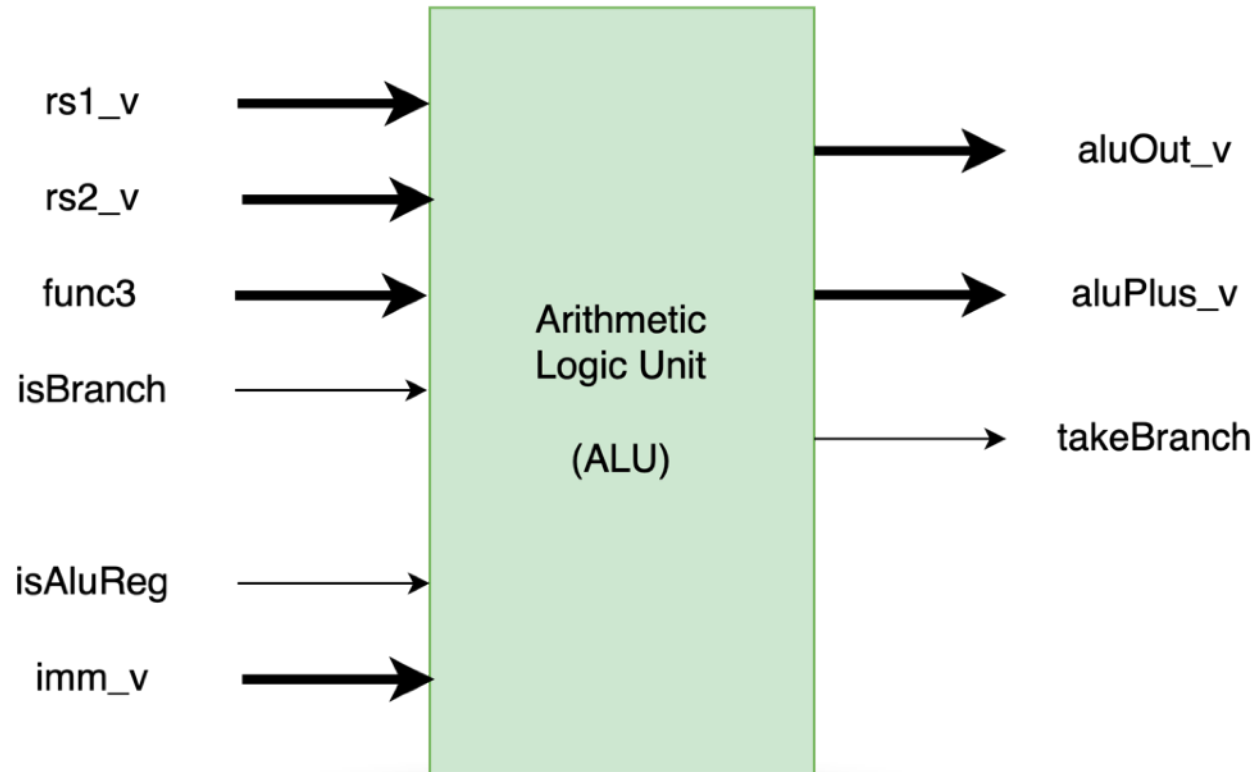
At the end of the execution stage, the data to be written to RD must be selected. This data can come from memory, the ALU or the CSRs.

The RAM module



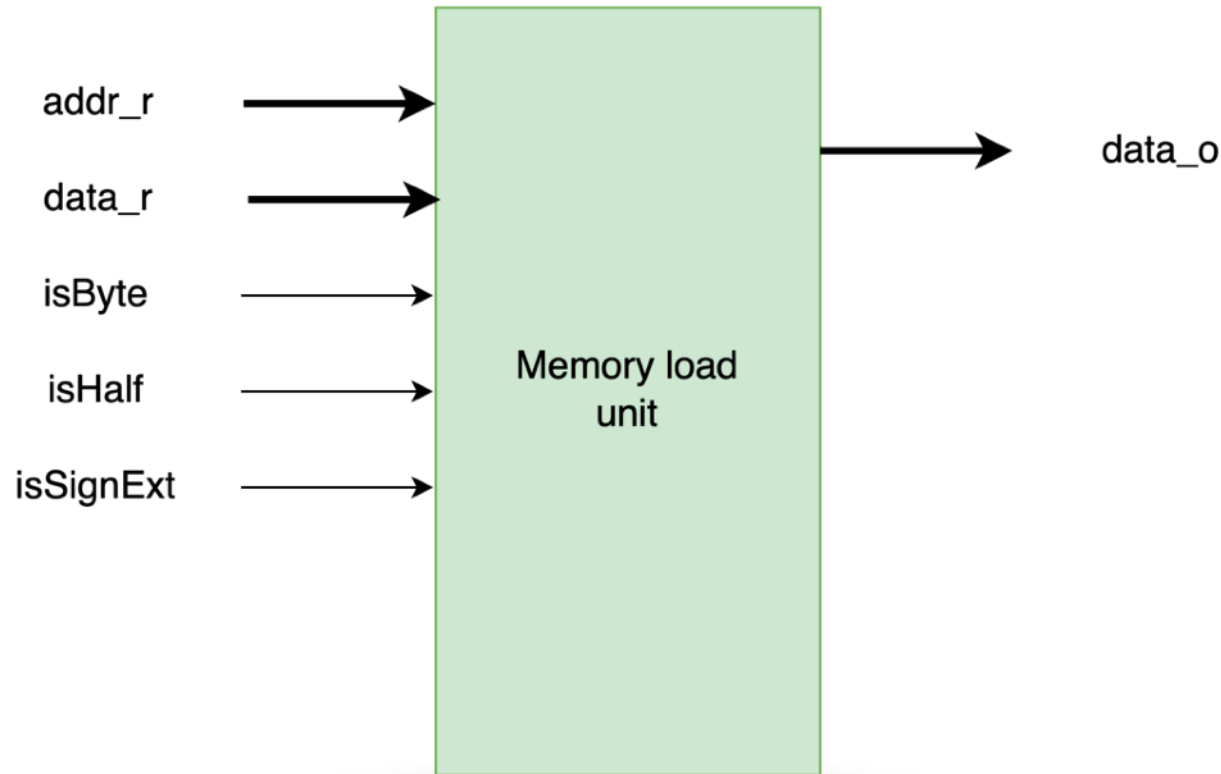
Data memory contains all the data that will be used in your program. Data width is 32 bits, but accesses (writes) can be on 8 bits, hence the write mask.

The Arithmetic Logic Unit



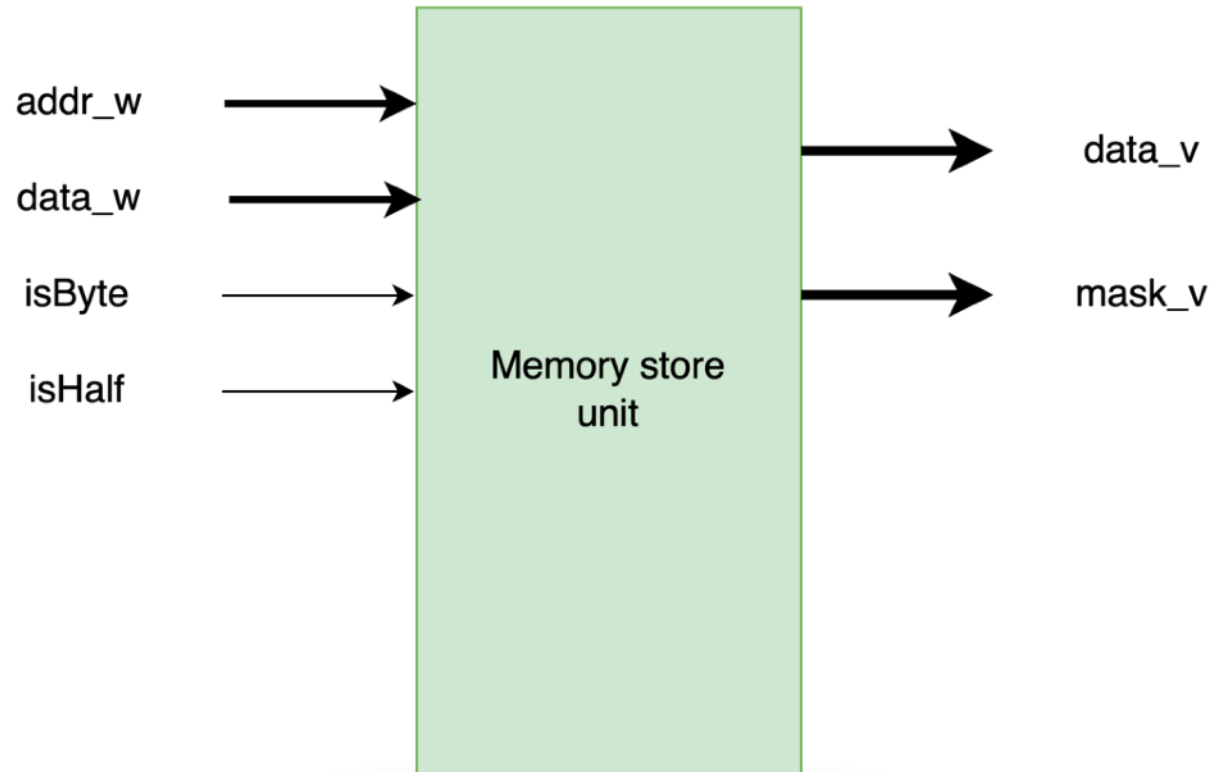
The arithmetic and logic unit is in charge of the actual calculations. This module is also in charge of branch prediction, including calculation of the new PC value.

The memory load module



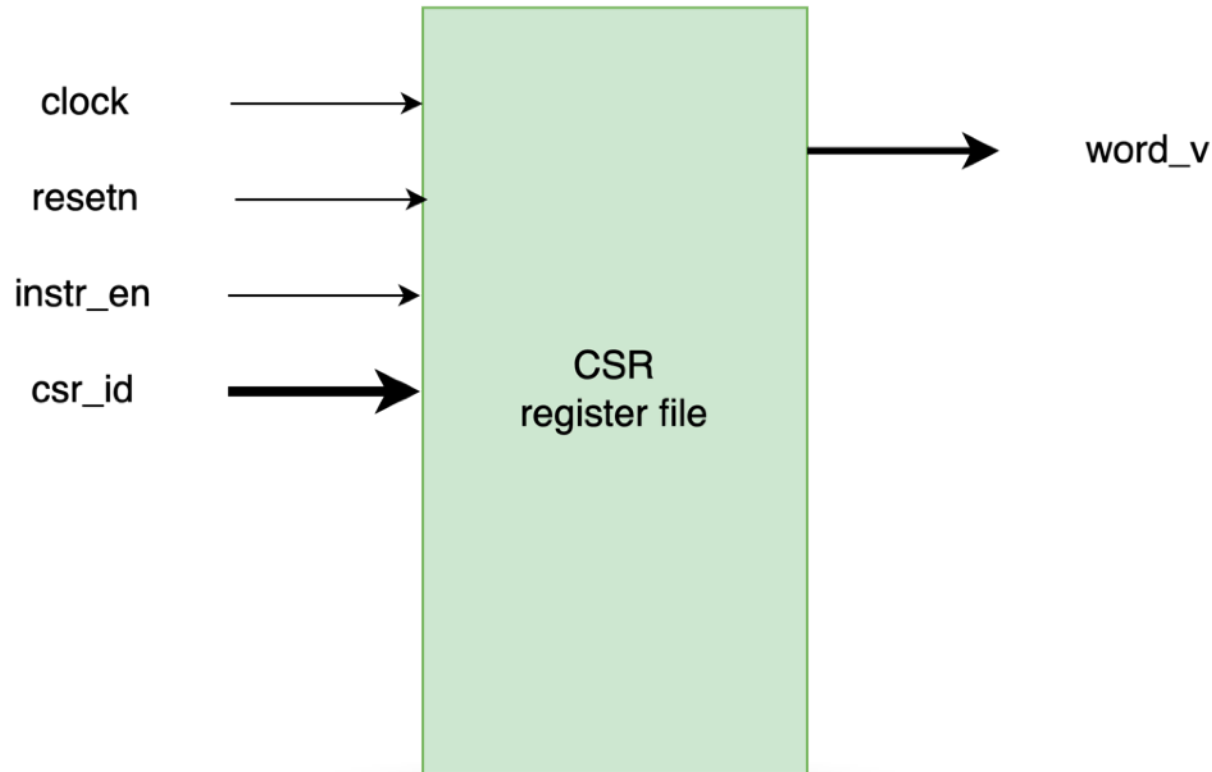
The load unit must realign the data coming from memory. This is because read accesses are always performed on 32 bits, even if the desired data is only `(u)int8_t`.

The memory store module



The store unit is the counterpart of the LOAD unit. It is responsible for preparing data for writing to memory. It generate a mask according to the data width.

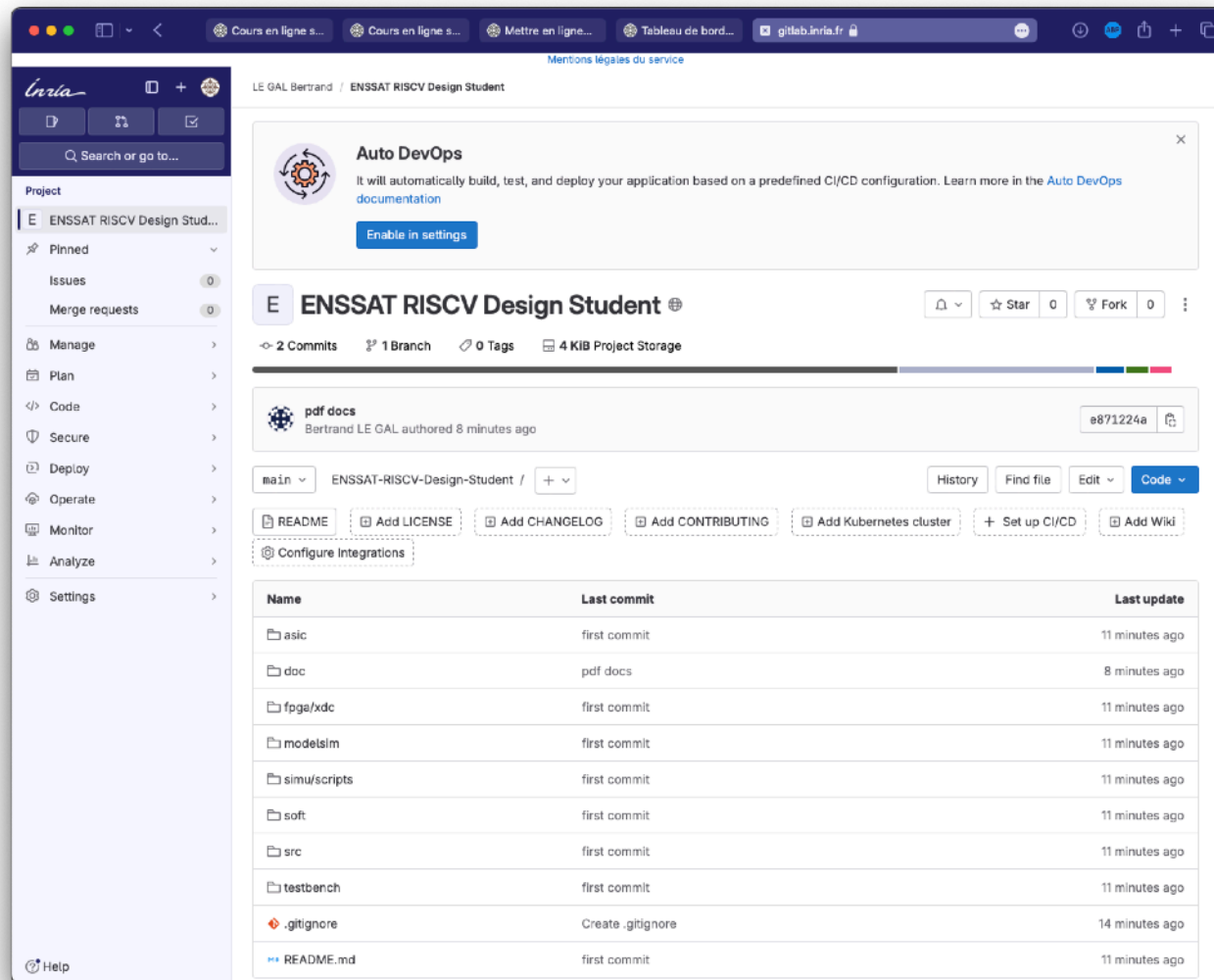
The CSR module



This complementary module is supplied with the system. It provides access to internal performance counters to (1) make architecture profiling and (2) setup some architectural features.

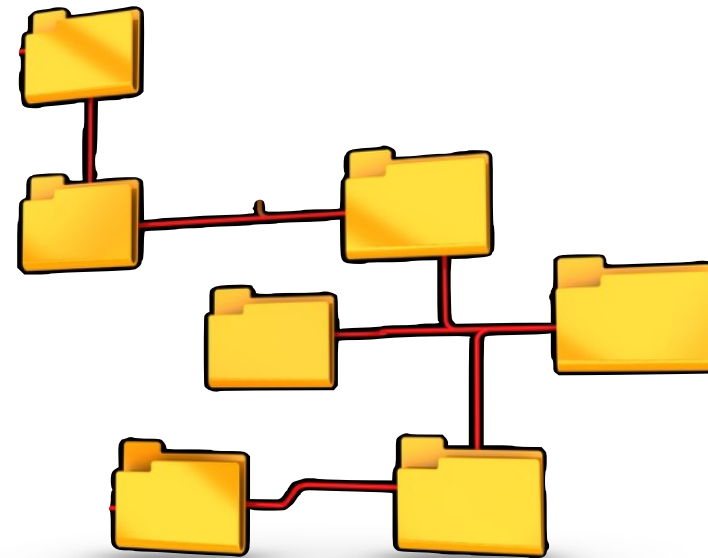
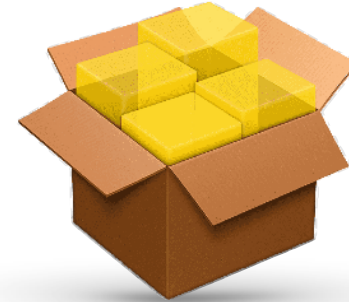
Provided framework

Downloading the project



`git clone https://gitlab.inria.fr/bertrand.le-gal/ENSSAT-RISC-V-Design-Student.git`

- ◎ The framework contains everything you need
 - ➔ VHDL source codes
 - ➔ C programs, etc.
- ◎ The package is composed of different sub-directories
 - ➔ src
 - ➔ doc
 - ➔ tbench
 - ➔ software
 - ➔ firmware
 - ➔ simu
 - ➔ modelsim
 - ➔ fpga
 - ➔ assis



© Let's have a look to « 0.5-hello » example

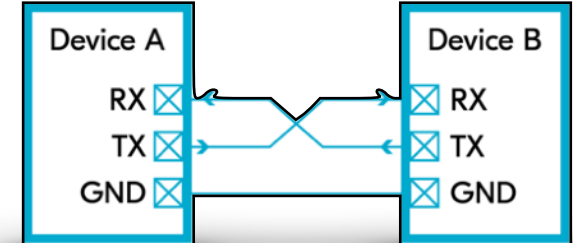
- ➔ It displays "Hello" on the screen
- ➔ Everything is contained in hello.c
- ➔ It couldn't be simpler!
- ➔ This is a basic "test" for your RISC-V

"Hello, World!" program handwritten in the [C language](#) and signed by [Brian Kernighan](#) (1978)

```
main()
{
    printf("hello, world\n");
}
```

© Highlights

- ➔ The UART is connected to the bus at 0x400008
- ➔ The origin of a program is the "start" function
- ➔ the "inline" keyword is used for copy and paste operations
- ➔ « **ebreak** » instruction stops the simulator (VHDL)



© Let's look at the assembler code

- ➔ Open the Makefile
- ➔ Compile the program with the "make" command
- ➔ Open the generated assembly code
- ➔ Open the "hex/firmware.mem" file



Command Line Interface